How Knowledge Updating Ripples into "Vulnerable" Knowledge?

Anonymous ACL submission

Abstract

Updating a language model's knowledge via fine-tuning or editing is necessary for keeping models up-to-date, yet it often triggers undesired ripple effects such as catastrophic forgetting or newly induced hallucinations, making the model output unreliable and cause negative effects on users. Since real-world knowledge is connected with each other, an important question is how such connectedness shapes the side effects of knowledge updates. We study this question by constructing a large-scale verified factual dataset from the real-world Wikipedia corpus, preserving both the natural distribution of triplets and their relational connections. We perform knowledge updates via fine-tuning and evaluate surrounding facts at varying relational distances before and after updating. Our results show that update-induced errors propagate far beyond the edited fact, with measurable effects even beyond five hops. Surprisingly, highly connected knowledge is more likely to flip from correct to incorrect, suggesting that familiar or widely connected knowledge is not necessarily more robust. hallucinations. Finally, a lightweight popular knowledge preservation strategy that anchors a small set of verified highly connected facts substantially reduces long-range error propagation.

1 Introduction

Large language models (LLMs) are powerful in storing and utilizing knowledge, yet their knowledge needs to continuously evolve with the changing world. This has led to widespread use of post-training techniques such as fine-tuning and knowledge editing *Heng* **[add citations]** to update model knowledge. However, updating a single piece of knowledge often introduces unintended side effects, where changes propagate beyond the target fact and distort other, seemingly unrelated knowledge. *Kathy* **[This is the continual learning problem and appropriate references should be inserted.]**This phenomenon, commonly referred to as the *ripple effect*, highlights a fundamental challenge: knowledge updating in LLMs is not a local operation, but can trigger global changes in an interconnected system.

While ripple effects have been observed, it remains unclear *what governs their propagation across knowledge*. A natural hypothesis is that such effects are driven by semantic or lexical similarity between facts. Prior work and empirical observations suggest that models are prone to confusion among semantically related entities, leading to the expectation that errors may propagate along similarity (cite). At the same time, another plausible hypothesis is that different types of knowledge exhibit different levels of robustness. In particular, long-tail knowledge, which is less frequently observed during training, is often assumed to be more fragile and thus more susceptible to corruption (cite). *Heng* **[these descriptions are all too abstract]** In this work, we aim to test these hypotheses and answer a central question: *Kathy* **[The above section is nicely written so long as there are ties to continual learning. The difference may be in looking at relatedness in language. I don't think semantic relatedness is generally considered, though we should check.]**

What determines the ripple effect in knowledge updating?

To systematically study how relationships between knowledge affect ripple propagation, we construct a structured knowledge graph grounded in real-world corpora. This allows us to explicitly model connections between facts and quantify how updates propagate along these connections. Within this framework, each piece of knowledge corresponds to a node, and relationships between facts define the edges, enabling us to analyze how ripple effects unfold along the graph structure.

Using this framework, we analyze how ripple effects vary across different parts of the graph. We observe that nodes with higher connectivity are significantly more vulnerable to corruption than sparsely connected ones, where connectivity re-

flects popularity of knowledge. Yuji[define popularity explicitly and accurately] Moreover, highly connected nodes not only exhibit greater susceptibility, but also play a dominant role in propagating errors to other parts of the knowledge space.

Interestingly, the connectivity of a node in the graph aligns with how frequently a piece of knowledge is referenced or shared across facts, allowing us to interpret it as a proxy for knowledge popularity. Kathy[OK I see it here. But use italics and more formally define. I think you do want to distinguish from frequency. I thought in Bengal talk you said it was not frequency.] Under this interpretation, highly connected nodes correspond to widely shared or commonly used knowledge, while sparsely connected nodes correspond to long-tail knowledge. From this perspective, the observed behavior is counterintuitive: widely shared (i.e., popular) knowledge is generally expected to be more robust, Heng[not sure what you mean by knowledge is robust] yet we find it to be more vulnerable and more influential in error propagation. This finding contradicts both the similarity-based explanation Kathy[what is this? explain and cite.]and the common intuition that long-tail knowledge is the primary source of fragility, Kathy[cite this about long-tailed also.]suggesting instead that ripple effects are governed by how centrally knowledge is connected. This observation naturally leads to a deeper question:

Why is hub knowledge both more vulnerable and more influential?

To answer this, we analyze the internal representations of LLMs from two complementary perspectives. First, we show that popular knowledge exhibits significantly narrower decision margins, indicating fragile decision boundaries that are easily perturbed by updates. Kathy[This previous sentence not clear. What are narrower decision margins?]Second, we demonstrate that hub nodes participate in stronger attention connectivity, forming effective pathways for propagating updates across distant knowledge. Together, these results suggest that the same structural property, connectivity, simultaneously determines both the stability of knowledge representations and the pathways through which errors spread. Finally, this understanding raises a practical question:

Can we leverage hub knowledge to control ripple effects?

If highly connected nodes amplify error propagation, they may also serve as anchors to stabilize the model. Building on this insight, we propose a connectivity-aware regularization strategy that selectively anchors hub knowledge during updates. We show that this approach effectively reduces long-range ripple effects and improves overall stability, outperforming connectivity-agnostic baselines. This demonstrates that the same mechanism that causes instability can also be harnessed for control.

Overall, our work reveals a unifying principle:

Node popularity, as a proxy for structural connectivity, governs both the vulnerability and propagation of ripple effects in LLMs.

By identifying connectivity as the key factor underlying knowledge distortion, we provide both a deeper understanding of knowledge dynamics in LLMs and a practical pathway toward more reliable knowledge updating.

Kathy[Overall this intro is very interesting. Just needs tightening up of terminology use, more citations, and definitions.]

2 Related Work

Yuji[In related work, we need to clarify several points: 1. the difference between expected ripple effect and unexpected ripple effect.)]

2.1 Knowledge Injection in LLMs

Knowledge updating has become a key approach to enhance LLMs' factuality, with methods spanning fine-tuning (Zhu et al., 2020), prompt engineering (Wei et al., 2022), and knowledge graph (KG) integration (Pan et al., 2023; Yasunaga et al., 2021).

Bosselut et al. (2019) fine-tuned LLMs with KG triples to improve question-answering accuracy, while Yasunaga et al. (2021) integrated KG embeddings into LLM layers to reduce hallucinations. Yuji[Revise this paragraph to emphasize that previous work focuses more on expected ripple effects and explores more about how similarity or logic affects ripple effects] However, most studies only evaluate post-injection performance via task accuracy (e.g., answer correctness) and rarely investigate how Weibing[controlled Weibing[artificial] updates] Yuji[not necessarily false, common knowledge updating also causes it.] distort model behavior—let alone their ripple effects on interconnected knowledge.

2.2 Continual Learning and Catastrophic Forgetting

Our investigation of ripple effects is fundamentally connected to the rich literature on Contin-

ual Learning (CL) and catastrophic forgetting (**??**). Traditional CL research focuses on sequential learning of new tasks without degrading performance on previously learned ones, proposing regularizers like Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017), episodic memory replay (e.g., GEM, A-GEM) (**??**), and architecture-expansion methods. While CL typically studies macroscopic performance degradation across entire datasets or task distributions, our work adapts this framework to the microscopic level: we use *Weibing* [artificial]injections as controlled, single-fact perturbations to trace exactly *how* catastrophic forgetting propagates along topological boundaries within the model's internal knowledge graph.

3 The RIPPLEEVAL Benchmark

To comprehensively analyze how factual updates affect related knowledge after post-training, we construct **RIPPLEEVAL**, a QA-based benchmark grounded in verified factual relations.

Each QA sample in RIPPLEEVAL corresponds to an underlying factual relation (s, r, o), where s and o are entities and r is a relation. We use these grounded relations to connect QA samples because QA expressions themselves are not stable units for defining factual dependency: the same fact can be asked in different ways, while unrelated facts may share similar question templates. Thus, the graph is used to define factual connections among QA samples, while model training and evaluation are still performed through natural-language QA.

Grounded Graph Construction. We construct an entity-level factual graph and use it to organize QA samples. In this graph, each node is an entity, and each directed edge represents a verified factual relation (s, r, o) from subject entity s to object entity o under relation r. Each verified edge is then verbalized into a QA sample for training and evaluation.

Graph Construction under Constrainted Relations. In ripple-effect evaluation, an edited fact is expected to change or preserve the answers to a set of related factual queries. To determine whether such downstream behavior is correct, each query must have an unambiguous target answer. This becomes problematic for one-to-many relations: for a subject–relation pair with multiple valid objects, a model prediction may differ from the annotated object while still being factually correct. Such ambiguity would make it unclear whether an apparent mismatch reflects an incorrect ripple effect or an incomplete annotation.

To avoid this issue, RIPPLEEVAL restricts evaluation to subject–relation pairs (s, r) with a unique or primary expected object o. Relations such as *CapitalOf* and *DevelopedByPrimary* satisfy this requirement, whereas relations such as *HasChild* are excluded because they may have multiple valid objects.

This design differs from prior benchmarks such as MQUAKE (Zhong et al., 2023) and RippleEdits (Cohen et al., 2023), which evaluate whether model edits lead to correct multi-hop or downstream consequences, but do not explicitly control the answer cardinality of each queried subject–relation pair. By enforcing this constraint, RIPPLEEVAL can construct controlled factual neighborhoods whose ripple-effect targets are uniquely specified.

Expansion and Verification. We construct the factual grounding graph $\mathcal{G}_{fact}$ with an automated expansion-and-filtering pipeline. Starting from high-confidence seed triplets, such as `("Minecraft", "DevelopedByPrimary", "Mojang Studios")`, we iteratively expand from the entities reached so far using a predefined set of factual relations. During expansion, we discard any triplet whose object entity already appears earlier on the same path. This removes cyclic dependencies, where a later hop points back to an ancestor entity, and ensures that retained paths represent genuine downstream factual connections. We then verify the retained triplets and remove cases with unsupported, ambiguous, or non-primary objects.

Scaling Up Knowledge Graph. Instead of relying on small-scale local models, we utilize the high-throughput DeepSeek API (`deepseek-chat`) to simultaneously propose candidate triplets and generate high-quality natural language questions. Scaling this rigorous generation process consumed approximately 309.2 million tokens (182.6M prompt, 126.6M completion). Crucially, the generated questions are hardcoded directly into the graph's edge attributes. This mechanism strictly eliminates prompt variance during evaluation: models are tested using the exact same question string before and after the update, ensuring failures stem from structural corruption rather than wording ambiguity. Each candidate is then verified against Wikidata (Vrandečić and Krötzsch, 2014) through an external validation module, and unverified triplets

are discarded. The final graph $\mathcal{G}_{fact}$ comprises $100,015$ unique entity nodes and $432,562$ verified relation edges spanning 39 strict relation types.

Factual Connectivity as Popularity. After constructing the verified factual graph, we use entity connectivity to approximate the popularity of factual knowledge in the benchmark. For a factual relation (s, r, o), we measure the connectivity of its object entity o by its in-degree, i.e., how many verified subject-relation facts point to the same object. This score does not measure how often a QA wording appears; instead, it measures how widely the answer entity is shared across different factual relations.

This definition gives each QA sample a grounded popularity score through its underlying factual relation. A sample whose answer entity is connected to many other facts is treated as involving a more widely shared factual anchor, while a sample whose answer entity has few incoming facts is treated as involving a narrower factual context. In this way, the graph connectivity serves as a proxy for the uneven distribution of real-world knowledge, where some entities are linked to many facts and others are much more isolated.

External Validation of the Popularity Proxy. Because in-degree is defined entirely within $\mathcal{G}_{fact}$, it is not immediately clear whether it tracks any externally observable notion of popularity. To verify that it does, we compare in-degree against two independent corpus-level signals on the QID-resolved subset of the graph: (i) the per-entity surface-form frequency in 200,000 English Wikipedia articles ($\approx$ 100M tokens, matched with an Aho–Corasick automaton), and (ii) the 2024 Wikipedia pageview total of the entity's article (user-agent only). After restricting to nodes for which all three signals are non-zero, 35,868 entities remain (59.9% of QID-resolved nodes). On this set, the pairwise log-log Spearman rank correlations are $\rho(\text{in-degree}, \text{wiki frequency}) = +0.308$, $\rho(\text{in-degree}, \text{pageviews}) = +0.235$, and $\rho(\text{pageviews}, \text{wiki frequency}) = +0.351$ (all $p \approx 0$; Figure 1). The three signals are therefore consistently and positively related, yet none reaches $\rho = 0.4$, indicating that they capture related but non-redundant aspects of factual prominence rather than the same underlying quantity.

The agreement is strongest at the head of the distribution: among the top-10 entities ranked by wiki frequency, 8 also fall in the top decile by in-degree, and United States (Q30) is the maximum on both axes. The disagreement is concentrated in two complementary regimes. First, entities that are densely interlinked in $\mathcal{G}_{fact}$ but rare in Wikipedia surface text (e.g., *ESPN*, *Dell Technologies*, *Assembly language*; each with in-degree ≥ 15 and wiki frequency $= 1$) are recognized as factual anchors by in-degree but not by raw corpus frequency. Second, entities with high public attention but sparse factual annotations in $\mathcal{G}_{fact}$ (e.g., recent public figures and events with large pageviews and small in-degree) are recognized by pageviews but not by in-degree. In both cases the disagreement is structural rather than noise: in-degree measures how many distinct verified factual relations resolve to an entity, which is the property our ripple-effect analyses depend on. We therefore retain in-degree as the popularity proxy in Sections 4–6, and treat wiki frequency and pageviews as external corroboration rather than as substitutes.

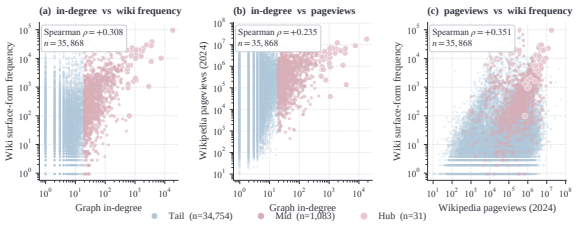

Figure 1: **External validation of the popularity proxy.** Pairwise log-log scatter of graph in-degree, 2024 Wikipedia pageviews, and entity surface-form frequency in 200k English Wikipedia articles, on the 35,868 QID-resolved nodes of $\mathcal{G}_{fact}$ for which all three signals are non-zero. Each panel reports the Spearman rank correlation. The three signals are consistently positively related (all $\rho > 0.2$) but none reaches $\rho = 0.4$, indicating non-redundant aspects of factual prominence. Hub, Mid, and Tail strata are defined by in-degree thresholds; hub points are plotted on top for visibility.

4 Experimental Setup

Having constructed RIPPLEEVAL, we use it to study how a controlled factual update affects connected factual knowledge in LLMs. Our experiments are designed around a single-edit setting: for each target fact, we independently update the model with a new to-be-update object and then evaluate how this update changes the model's behavior on related QA samples. This setup allows us to isolate the effect of one factual update while avoiding the confounding effects of large-scale continual training.

4.1 Subject Models

We evaluate three representative open-weight 7B models with different architectures and instruction-tuning properties:

- **Llama-2-7B-Chat** (**?**): a widely used chat-tuned baseline.
- **Mistral-7B-v0.3** (Jiang et al., 2023): a stronger 7B model with improved attention and context-handling mechanisms.
- **Qwen2.5-7B-Instruct** (Bai et al., 2023): a recent instruction-tuned model with strong reasoning and coding performance.

Using multiple model families allows us to test whether the observed ripple effects are specific to one model or consistently appear across different 7B-scale LLMs.

4.2 Updating Target Selection

For each experiment, we select a target factual relation (s, r, o) from the verified factual graph $\mathcal{G}_{fact}$. Each selected target is associated with the in-degree of its object entity o, which provides the factual connectivity score defined in Section 3. This score is used to organize target updates in later analyses.

The base model is reset before each target update. Therefore, each run measures the effect of a single factual update from the same pre-update model state, rather than the accumulated effect of multiple edits.

4.3 Updated Knowledge Format

For each selected target fact (s, r, o), we construct an update by replacing the original object o with a plausible but structurally distinct target object o'. For example, a target fact such as *CapitalOf(France, Paris)* may be updated to *CapitalOf(France, Lyon)*. We use this substitution to create a controlled perturbation that is unlikely to be already stored as the model's factual knowledge. This allows us to isolate how an injected update affects connected factual knowledge, rather than conflating the effect with the model's prior memory of real-world facts or uncontrolled real-world logical associations.

To train the model on the target update, we generate QA-format instruction pairs that express the new relation (s, r, o'). For each target fact, we first generate 30 diverse QA templates and then apply paraphrase augmentation to expand them into 150 targeted update pairs. These pairs serve as the direct supervision for injecting the target object o'.

To reduce overfitting to the target update and preserve unrelated model behavior, the final update set contains 650 QA pairs:

- 150 targeted update pairs expressing the injected fact (s, r, o');
- 400 neutral factual QA pairs sampled from distant, disconnected subgraphs in $\mathcal{G}_{fact}$;
- 100 out-of-domain irrelevant QA pairs, disjoint from the irrelevant evaluation set.

The generated QA prompts are fixed and reused before and after updating. This ensures that observed behavioral changes are not caused by prompt variation.

4.4 Update Optimization via Finetuning

We implement the factual update using Low-Rank Adaptation (LoRA) (Hu et al., 2021). We use parameter-efficient tuning rather than full finetuning because our goal is to study how a small factual update propagates through the model's existing knowledge representations, rather than to overwrite the model globally. LoRA provides a natural gradient-based update while keeping most model parameters frozen, making it suitable for measuring ripple effects under limited parameter disturbance.

We train LoRA adapters on the update set by minimizing the cross-entropy loss of the injected object o' under the target relation:

$$\min_{\theta} \mathcal{L}_{CE}(P_\theta(o' \mid s, r)).$$

Unless otherwise specified, we use LoRA rank $r = 20$, scaling factor $\alpha = 40$, dropout $p = 0.1$, and apply LoRA to the `q_proj` and `v_proj` matrices. We optimize with AdamW (**?**), using learning rate 2.5×10^{-4}, batch size 4, and 8 epochs, stopping after convergence when the training loss falls below 10^{-3}. Additional prompt templates and implementation details are provided in the Appendix.

Updating Success Evaluation: Injection Success Rate (ISR). Before analyzing unintended changes to neighboring facts, we first verify whether the update succeeds on the target fact itself. For an edit that replaces the original object t with the injected target t', ISR checks whether t'

receives the highest candidate score for the edited query $q_{h,r}$:

$$\text{ISR} = \mathbb{I}\left[\arg\max_{y \in \mathcal{C}_{h,r}} S_\theta(y \mid q_{h,r}) = t'\right], \quad (1)$$

where $\mathcal{C}_{h,r}$ is the candidate object set for the subject–relation query. We report the number of failed injections and compute downstream metrics both over all attempted edits and over the successful-edit subset, separating target-injection failures from unintended changes to related non-target facts.

4.5 Preserving General Model Performance

Because our goal is to study localized ripple effects without causing model collapse, we perform a global sanity check after each update. Specifically, we evaluate the edited model on a fixed set of 50 irrelevant factual questions that are disjoint from the target neighborhood and the update data. A valid update should preserve performance on this irrelevant set, indicating that the model has not undergone broad catastrophic forgetting.

This sanity check separates local ripple effects from global capability degradation. If the model fails broadly on unrelated questions, downstream errors cannot be confidently attributed to propagation through the factual graph. In our trials, accuracy on this irrelevant set remains stable, suggesting that the observed changes primarily reflect update-induced effects on connected factual knowledge rather than general model degradation.

Knowledge Distortion Evaluation. After verifying that the target update succeeds, we evaluate whether the update changes other factual QA samples connected to the edited fact in $\mathcal{G}_{fact}$. For each update, we sample up to 30 non-target neighbors from each hop distance $d \in \{1, \ldots, 5\}$. We retain only the queries that the model answers correctly before the update, since facts that are already incorrect cannot provide a clean signal of update-induced corruption.

We then query the updated model with the same fixed question string and compare its answer with the original verified factual answer. A non-target fact is counted as distorted if it was answered correctly before the update but fails alias-normalized exact matching after the update. We report the distortion rate as the fraction of retained non-target facts that undergo this correct-to-wrong change. Yuji[Formally define flip rate here]

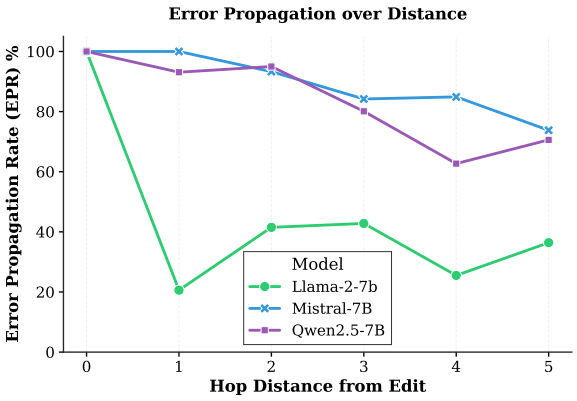

Figure 2: Weibing[Extent of Error Propagation.] Error Propagation Rate (EPR) across different hop distances ($d = 1$ to $d = 5$). Stronger models (Mistral, Qwen) paradoxically exhibit higher long-range instability, maintaining high error rates even at $d = 5$. Duo[We could provide sample counts per distance, confidence intervals, or statistical tests]

5 Experimental Results

We use RIPPLEEVAL to study how a localized factual update affects connected non-target knowledge. Our experiments are organized around six research questions:

RQ1: Do factual updates produce non-local changes in connected knowledge?

RQ2: Are affected facts equally sensitive to these changes?

RQ3: Can corrupted non-target knowledge further propagate errors?

RQ4: Can knowledge similarity explain the observed ripple pattern?

RQ5: Can the observed propagation pattern guide more effective preservation?

RQ6: What internal changes are associated with the observed ripple pattern?

5.1 Undesired Ripples Exists in Long Distance

We begin with the first question: whether the side effects of a factual update remain local or the effect persists across multiple related hops. To test this, we use the correct-to-wrong flip rate of knowledge to measure error propagation over non-target facts at hop distances $d = 1, \ldots, 5$ from the edited fact. For each distance, we retain only facts that the model answered correctly before the update, so that evaluation captures update-induced correct-to-wrong changes rather than pre-existing model errors.

Figure 2 shows that factual updates produce clear non-local ripple effects. Error propagation remains

measurable across multiple hops, and in several models the distortion persists even at $d = 5$. This indicates that a single localized update can affect facts that are several relational steps away from the target, suggesting that update-induced errors can travel through connected factual knowledge rather than stopping at the edited fact.

Table 1: **Broad Impact of Semantic Similarity.** In our large-scale (47k) evaluation, while highly similar entities ($0.7 - 1.0$) have a high probability of flipping, they represent a very small portion of the dataset. Most errors occur in the low-similarity range.

Similarity Range	Count	Flip Rate (%)	Observation
$0.0 - 0.2$ (Low)	14,052	9.55%	**Dominant Failure Mode**
$0.2 - 0.4$ (Mid)	26,171	12.23%	Frequent Propagation
$0.4 - 0.6$ (High)	5,168	19.89%	Moderate Correlation
$0.7 - 1.0$ (Exact)	713	**36.46%**	High Risk, Rare Occurrence

5.2 Popular Knowledge Is More Vulnerable to Updates

Having shown that factual updates can produce long-range ripple effects, we next ask whether all connected facts are equally affected. A natural hypothesis is that long-tail knowledge could be more fragile, as prior work has shown that models are more prone to hallucination or factual errors on less frequent knowledge Yuji[cite]. We therefore examine this question by grouping the evaluated facts by the in-degree of their grounded object entity, following the factual popularity definition in Section 3. High-Popularity facts correspond to widely shared answer entities, while Low-Popularity facts correspond to sparsely connected answer entities.

Direct vulnerability. As shown in Figure 6(a), Counter-intuitively, high-Popularity facts are more likely to flip from correct to incorrect after a related update. At the immediate-neighbor distance ($d = 1$), High-Popularity facts have a Flip Rate of 33.3%, whereas Low-Popularity facts have a Flip Rate of 16.0%. This indicates that facts grounded in highly connected answer entities are not necessarily more stable; instead, they are more easily overturned when nearby factual knowledge is modified.

Neighbor vulnerability. We further test whether this vulnerability persists when popular facts are not the edited source, but only neighboring facts affected by the update. As shown in Figure 7, High-Popularity neighbors experience a larger accuracy drop regardless of the source type. Notably, even when the update targets a Low-Popularity source, High-Popularity neighbors suffer an accuracy drop of approximately 8.8%, substantially higher than the drop observed for Non-Hub neighbors, approximately 3.4%. This result suggests that popular knowledge is especially sensitive to collateral changes: it can be affected not only when it is directly involved in an update, but also when updates occur in its factual neighborhood.

View	Similarity Bin	Count	C→W Rate	Flip Share	Margin Share
Raw	Low (< 0.25)	1,369	14.32%	55.52%	59.91%
	Mid ($[0.25, 0.5)$)	1,096	13.96%	43.34%	39.64%
	High (≥ 0.5)	35	11.43%	1.13%	0.44%
Mask B	Low (< 0.25)	496	29.03%	58.78%	61.34%
	Mid ($[0.25, 0.5)$)	389	25.45%	40.41%	38.19%
	High (≥ 0.5)	8	25.00%	0.82%	0.47%

Table 2: **Paired-Compatible Semantic Control.** Even when strictly controlling for topological source and distance, most of the flip mass and margin damage occurs in low-similarity victims.

5.3 Popular Knowledge Causes Wider Error Propagation

Yuji[My second-round revision progress bar is here...] We next examine whether factual popularity affects the impact of an update when the popular fact is used as the update source. While the previous section shows that High-Popularity facts are more vulnerable as affected neighbors, this section asks whether updating a High-Popularity fact also causes wider downstream damage.

As shown in Figure 6(b), updates targeting High-Popularity facts lead to substantially more error propagation than updates targeting Low-Popularity facts. This result indicates that High-Popularity facts act as stronger propagators of knowledge corruption.

5.4 Surface Similarity Does Not Explain Most Ripple Effects

A natural alternative explanation is that the observed ripple effects are caused by surface-level confusion rather than factual connectivity. For example, if an update about *Apple Inc.* causes a flip in a fact about *Apple Corps*, the error may arise because the entity names are similar, not because the two facts are connected through factual relations. To examine this possibility, we measure the normalized string similarity between the edited source entity and the affected neighbor entity using the Levenshtein ratio, and conduct two complementary analyses.

Broad similarity analysis. We first analyze approximately 47,000 source-neighbor pairs across the full evaluation set. As shown in Table 6, entity pairs with very high string similarity (> 0.7) do have a higher Flip Rate, around 36%. However, such pairs are rare and account for less than 2% of all evaluated pairs. In contrast, more than 95% of ripple errors occur in pairs with low or moderate similarity (< 0.4). The overall Pearson correlation between string similarity and binary flip status is also weak ($\rho = 0.12$). These results suggest that surface similarity can increase risk in a small subset of cases, but it does not explain the majority of observed flips.

Paired control within the same factual neighborhood. We further conduct a stricter paired analysis by comparing neighbors from the same update source and the same hop distance. This controls for the edited fact and the distance from the update, allowing us to test whether more similar entities are more likely to flip within the same factual neighborhood.

Under this paired setting, string similarity is not a reliable predictor of factual flips. The mean Pearson correlation between similarity and flip status is close to zero ($r \approx -0.02$). As shown in Table **??**, high-similarity neighbors are also extremely sparse, accounting for less than 1.5% of the paired samples. Even under the clean-correct mask, where the model initially predicts the correct answer at rank 1, low-similarity neighbors still account for 58.78% of all Correct-to-Wrong flips and 61.34% of the total margin-loss mass.

Together, these results show that surface similarity is not the main driver of the ripple effects we observe. Although similar entity names can introduce local confusion, most errors occur between entities with low surface similarity. This supports our interpretation that factual connectivity, rather than question wording or entity-name similarity alone, is necessary for understanding how update effects spread across related knowledge.

5.5 Popularity Anchoring Mitigates Ripple Effects

The previous sections show that high-popularity facts are both more vulnerable to neighboring updates and more influential when used as update sources. This raises a practical question: can high-popularity facts also be used to stabilize knowledge updating? To answer this question, we test a mitigation strategy called **Popularity Anchoring**. The key idea is to preserve the model's behavior on a small set of high-popularity factual anchors while updating the target fact.

During the update, we optimize the model with two objectives. The first objective is the standard cross-entropy loss $\mathcal{L}_{CE}$ for learning the injected fact. The second objective is a KL-divergence regularizer that keeps the edited model close to the original model on a small set of anchor prompts $\mathcal{A}$:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda \sum_{x \in \mathcal{A}} D_{KL}\left(P_{clean}(\cdot \mid x) \,\|\, P_{edited}(\cdot \mid x)\right),$$

where P_{clean} is the output distribution of the original model, P_{edited} is the output distribution of the edited model, and λ controls the regularization strength. We set $\lambda = 0.1$ in all experiments.

For **Popularity Anchoring**, the anchor set $\mathcal{A}$ consists of QA prompts whose answer entities have high factual popularity, measured by object in-degree. To prevent test-set leakage, anchor prompts are drawn from factual regions that share no paths with the target update neighborhood. We compare Popularity Anchoring with two baselines: **No Anchoring**, which performs the update without the KL regularizer, and **Random Anchoring**, which uses the same number of randomly sampled anchor prompts.

Comparison with Random Anchoring. Table 7 compares the accuracy drop after updating under different anchoring strategies. Random Anchoring provides stronger protection in the immediate neighborhood ($d = 1$), reducing the accuracy drop to -16.5%. However, Popularity Anchoring becomes more effective at larger distances. For example, at $d = 4$, Popularity Anchoring limits the accuracy drop to -10.3%, compared with -13.6% for Random Anchoring. This suggests that high-popularity anchors are especially useful for suppressing longer-range ripple effects.

Sample Efficiency. We further vary the number of anchor prompts N. As shown in Figure 3, Popularity Anchoring achieves stronger data efficiency than Random Anchoring. With only $N = 25$ anchor prompts, it recovers a substantial portion of the performance loss. With $N = 100$, it nearly eliminates the average accuracy drop and even produces a slight positive change ($+0.6\%$). These results suggest that selecting anchors by factual popularity is more effective than randomly select-

ing factual prompts, especially when the number of anchors is limited.

5.6 Mechanistic Explanations: Margin and Attention

To understand *why* Hubs are more vulnerable and cause wider error propagation, we delve into the models' internal representations, examining static decision boundaries (Logit Margin) and dynamic propagation pathways (Attention Lift).

Static Vulnerability: Narrow Decision Boundaries. First, we evaluate the pre-update decision boundaries (Logit Margin) of Hubs versus Tail entities. To ensure a rigorous baseline, we apply a strict filter (Mask B), retaining only samples where the clean model accurately predicts the correct token at rank 1 (`clean_accuracy == 1` and `clean_correct_token_rank == 1`). We find that the average clean margin for Hubs is significantly lower than that of Tail nodes. Because Hub entities heavily co-occur with numerous other entities in the pre-training corpus, their feature spaces are highly crowded. Consequently, their decision boundaries are thinner and more fragile, explaining why a minimal gradient update easily pushes them across the threshold into an incorrect prediction (as observed in Figure 6a).

Dynamic Propagation: *Weibing* [Attention Shift across the Graph.] Next, we analyze how errors propagate by measuring the change in Attention Lift ($|\Delta$Attention Lift$|$) targeted at k-hop neighbors post-update. We use the fully populated Llama-2 paired audit ($n = 30$ evaluation samples per hop) as the primary evidence source and report the last-layer, first-generation-step attention lift defined in Section 4.

Method	d1	d2	d3	d4	d5
Baseline	-43.2	-15.5	-20.9	-37.6	-23.3
Random Anchor	**-16.5**	-5.2	-3.6	-13.6	-10.4
Hub Anchor	-39.2	**-5.5**	**-2.1**	**-10.3**	**-8.5**

Table 3: **Mitigation Performance**. Values indicate the drop in accuracy relative to the pre-edit state. Hub Anchoring shows better stability at larger distances ($d \geq 3$).

Table 4 makes the attention pattern explicit. At $d1$–$d3$, the hub-sourced update produces larger attention perturbations than the tail-sourced update (e.g., $0.07246 > 0.06912$ at $d1$ and $0.05970 > 0.04318$ at $d3$). At $d4$–$d5$, this pattern reverses, with the tail-sourced update exhibiting a clear rebound in attention lift ($0.10443 > 0.04287$ at $d4$ and $0.09825 > 0.04901$ at $d5$).

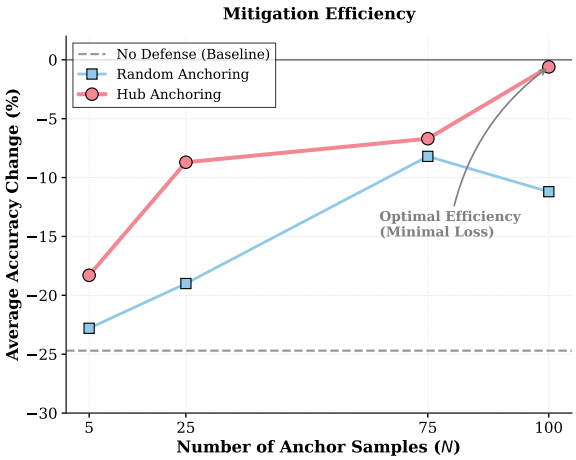

Figure 3: **Data Efficiency Analysis.** *Yuji* [Add a longtail anchoring ablation here.] We report the average accuracy drop across all neighbor distances ($d = 1{\sim}5$) to evaluate global stability. **Hub Anchoring** demonstrates superior efficiency: with only $N = 25$ samples, it recovers most of the performance (-8.7% vs. -24.7% baseline), and at $N = 100$, it achieves positive knowledge consolidation ($+0.6\%$).

Hop	Hub $\lvert\Delta$AttLift$\rvert$	Tail $\lvert\Delta$AttLift$\rvert$	Hub > Tail?
d1	0.07246	0.06912	Yes
d2	0.05836	0.04996	Yes
d3	0.05970	0.04318	Yes

Table 4: **Attention Lift Perturbation by Hop in the Llama-2 Paired Audit.** Absolute post-update attention-lift change ($|\Delta$AttLift$|$) for the hub-sourced update (006) and the tail-sourced update (007), measured on the paired audit with $n = 30$ evaluation samples per hop. The statistic is computed from the final transformer layer at the first generated token step, averaged over all heads and query positions, and evaluated on the matched neighbor head-token span. Hubs show larger perturbations at $d1$–$d3$.

- **Immediate Neighborhood Propagation ($d \in [1, 3]$):** In the immediate to mid-range neighborhood, Hubs usually exhibit larger attention lift perturbations and larger $|\Delta$margin$|$ fluctuations than Tails. This supports the structural hypothesis that dense hub connectivity acts as an efficient conduit, disproportionately propagating updated representations to nearby nodes.

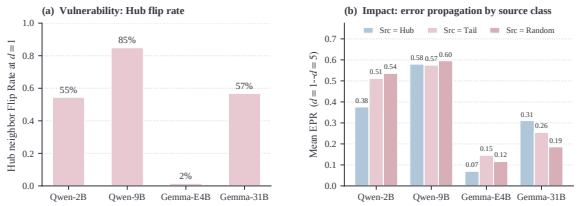

Figure 4: **The Popularity Paradox.** (a) **Vulnerability:** High-popularity nodes (Hubs) are significantly more likely to flip from Correct to Wrong (33.3%) compared to tail nodes (16.0%) when a neighbor is edited. (b) **Impact:** *Weibing* [When Hubs are the source of an update, they act as central propagators, causing drastically higher error propagation rates across all models.]

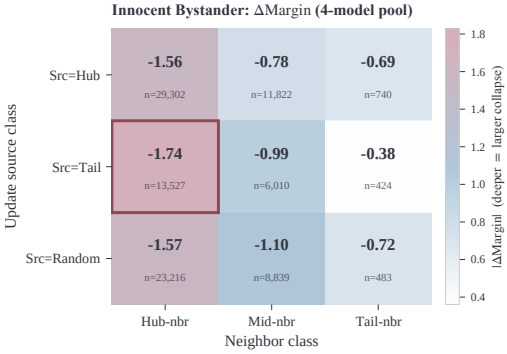

Figure 5: **The Innocent Bystander Effect.** Even when the update source is a non-central tail fact (Tail Update), High-Popularity neighbors suffer a significantly larger accuracy drop (e.g., 8.8% vs 3.4% on Mistral) compared to other neighbors. Hubs act as the primary collateral damage.

Table 5: **Hub-neighbor margin collapse is consistent across all four evaluated model families.** Mean correct-token margin change $\Delta\text{Margin} = \text{Margin}_{\text{post}} - \text{Margin}_{\text{pre}}$, pooled over d=1 to d=5 on the Mask-B subset (pre-update-correct facts; see Section 4). More negative = deeper confidence collapse. The Hub-neighbor column suffers the deepest drop in **4/4** models (Hub–Tail gap shown in the last column). The signal also persists on the heavily instruction-tuned Gemma-4-E4B-it, where output-level flip behaviour is otherwise heavily suppressed.

Model	Hub	Mid	Tail	Hub − Tail
Qwen3.5-2B	**−0.90** (n=8,026)	+0.05 (n=2,901)	−0.25 (n=199)	**−0.65**
Qwen3.5-9B	**−2.18** (n=10,956)	−1.80 (n=5,253)	−1.64 (n=322)	**−0.55**
Gemma-4-E4B-it	**−0.33** (n=8,791)	+0.09 (n=2,753)	+0.48 (n=179)	**−0.81**
Gemma-4-31B-it	**−6.65** (n=11,703)	−5.12 (n=5,672)	−3.79 (n=356)	**−2.86**
Hub > Tail collapse		**4 / 4 models**		

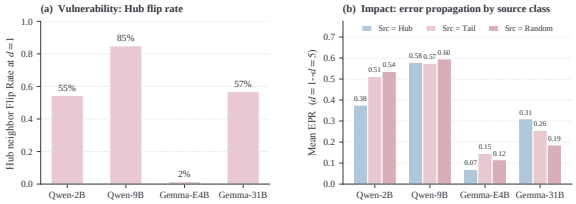

Figure 6: **The Popularity Paradox.** (a) **Vulnerability:** On Qwen3.5-9B, High-Popularity nodes (Hubs) suffer the largest correct-token margin collapse under nearby updates, and lose their top-1 prediction at d=1 at a rate of 84.9%; the deeper ΔMargin signal on Hub neighbors holds in 4/4 evaluated families (Qwen3.5-2B/9B, Gemma-4-E4B-it/31B-it) — see §5.2. (b) **Impact:** When Hubs serve as the update source, they act as central propagators, producing substantially higher long-range Error Propagation Rate across all evaluated models — Qwen3.5-9B reaches mean EPR = 0.580 across d=1–d=5 and still 0.515 at d=5.

6 Remaining figures and tables

Limitations

Weibing [While our study provides systematic insights into knowledge updating, we acknowledge several limitations that should be addressed in future work. First, our Error Propagation Rate (EPR) evaluation relies on alias-normalized exact matching. Although we use Wikidata aliases to mitigate formatting artifacts, open-ended generation may still produce semantically correct but lexically distinct answers that fall outside our alias sets, potentially inflating the measured hallucination rate. Second, our constructed RIPPLEEVAL knowledge graph enforces a strict N-to-1 Directed Acyclic Graph (DAG) topology to ensure unambiguous downstream paths. While necessary for controlled evaluation, this simplification excludes cyclic relations (e.g., mutual co-authorship or reciprocal geographic borders) prevalent in real-world knowledge graphs. Finally, our experimental design utilizes one-shot *Weibing* [artificial] injections to cleanly trace topological propagation. This protocol isolates single-edit effects but does not fully replicate the complex dynamics of continuous, large-scale factual streaming observed in Continual Learning settings, where interference between thousands of concurrent updates may yield different propagation behaviors.]

Acknowledgments

This document has been adapted by Steven Bethard, Ryan Cotterell and Rui Yan from the instructions for earlier ACL and NAACL proceedings, including those for ACL 2019 by Douwe Kiela and Ivan Vulić, NAACL 2019 by Stephanie Lukin and Alla Roskovskaya, ACL 2018 by Shay Cohen, Kevin Gimpel, and Wei Lu, NAACL 2018 by Margaret Mitchell and Stephanie Lukin, BibTEX suggestions

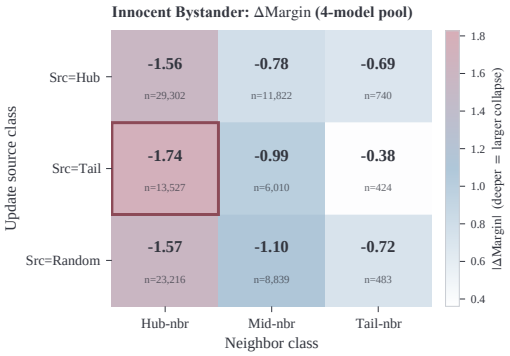

Figure 7: **The Innocent Bystander Effect.** Even when the update source is a non-central Tail fact (*Tail Update*), High-Popularity (Hub) neighbors experience the largest *margin collapse*. Pooled across four models on 94,363 Mask B facts, the correct-token logit margin drops by -1.74 in the Tail-source $\rightarrow$ Hub-neighbor cell, versus -0.38 in the Tail-source $\rightarrow$ Tail-neighbor cell and -0.69 in the Hub-source $\rightarrow$ Tail-neighbor cell. Hubs act as the primary collateral damage.

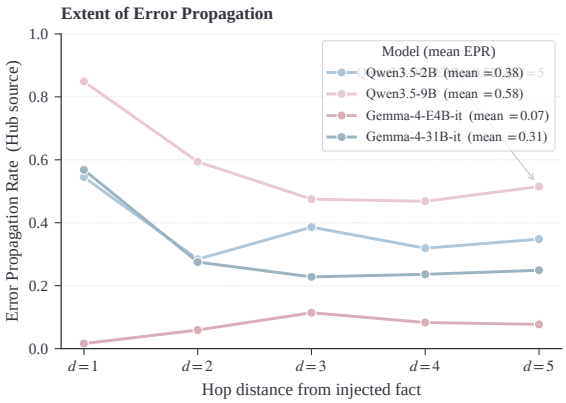

Figure 8: **Extent of Error Propagation.** Error Propagation Rate (EPR, Section 4, Eq. **??**) across hop distances d=1 to d=5 for the four evaluated model families. Larger Qwen models exhibit higher long-range instability (Qwen3.5-9B maintains EPR $= 0.515$ at d=5 under Hub-source updates), while the small instruction-tuned Gemma-4-E4B-it is the most resistant.

Table 6: **Broad impact of surface similarity (four-model Mask B pool).** In the 94,363 Mask B source-neighbor pairs pooled across the four evaluated models, highly similar entity pairs (≥ 0.8) flip at $\sim$50% but represent under 1% of the data. Most errors occur in the low-similarity range with a near-constant flip rate.

Similarity Range	Count	Flip Rate (%)	Observation
$[0.0, 0.2)$ (Low)	24,512	28.12%	**Dominant Failure Mode**
$[0.2, 0.4)$ (Mid)	60,742	29.19%	Frequent Propagation
$[0.4, 0.6)$ (High)	7,747	32.85%	Moderate Correlation
$[0.6, 0.8)$	475	39.16%	High Risk, Rare
$[0.8, 1.0]$ (Exact)	887	**49.94%**	High Risk, Rare Occurrence

for (NA)ACL 2017/2018 from Jason Eisner, ACL 2017 by Dan Gildea and Min-Yen Kan, NAACL 2017 by Margaret Mitchell, ACL 2012 by Maggie Li and Michael White, ACL 2010 by Jing-Shin Chang and Philipp Koehn, ACL 2008 by Johanna D. Moore, Simone Teufel, James Allan, and Sadaoki Furui, ACL 2005 by Hwee Tou Ng and Kemal Oflazer, ACL 2002 by Eugene Charniak and Dekang Lin, and earlier ACL and EACL formats written by several people, including John Chen, Henry S. Thompson and Donald Walker. Additional elements were taken from the formatting instructions of the *International Joint Conference on Artificial Intelligence* and the *Conference on Computer Vision and Pattern Recognition*.

References

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenhang Ge, Yu Han, Fei Huang, and 1 others. 2023. Qwen technical report. In *ArXiv*. ArXiv ID: 2309.16609.

Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Celikyilmaz, and Yejin Choi. 2019. Comet: Commonsense transformers for automatic knowledge graph construction. In *Annual Meeting of the Association for Computational Linguistics*. ArXiv ID: 1906.05317.

Roi Cohen, Eden Biran, Ori Yoran, A. Globerson, and Mor Geva. 2023. Evaluating the ripple effects of knowledge editing in language models. In *Transactions of the Association for Computational Linguistics*. ArXiv ID: 2307.12976.

J. E. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. In *ArXiv*. ArXiv ID: 2106.09685.

Albert Qiaochu Jiang, Alexandre Sablayrolles, A. Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. In *ArXiv*. ArXiv ID: 2310.06825.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, and Demis Hassabis. 2017. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences (PNAS)*, volume 114, pages 3521–3526.

Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying large language models and knowledge graphs: A roadmap. In *IEEE Transactions on Knowledge and Data Engineering*. ArXiv ID: 2306.08302.

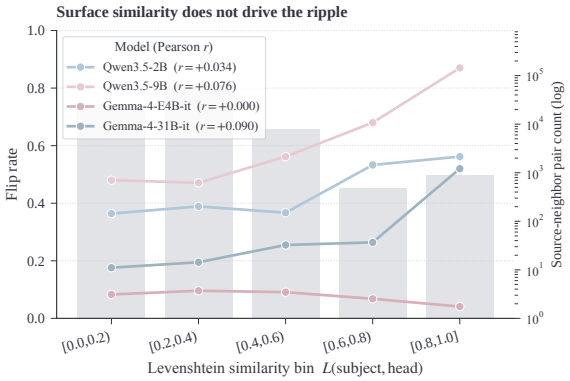

Figure 9: **Flip Rate vs subject-to-head Levenshtein similarity, per model.** For each of the four evaluated models, the Flip Rate is roughly flat across the dominant similarity bins ($L < 0.6$, which contain $> 98\%$ of all 94,363 source-neighbor pairs; gray bars on the right axis) and only rises in the rare high-similarity tail. The per-model Pearson correlations $r(L_{\text{subject,head}})$ remain near zero (≤ 0.090), showing that lexical proximity is not the principal driver of the ripple effect.

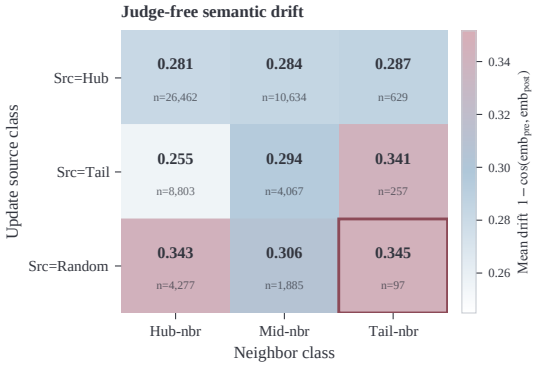

Figure 10: **Judge-free semantic drift across (update source $\times$ neighbor) cells.** Mean cosine drift $1 - \cos(\text{emb}(y_{\text{pre}}), \text{emb}(y_{\text{post}}))$ between pre- and post-update responses on 57,111 Mask B facts (pooled across the four models). Drift values are tightly clustered in $[0.255, 0.345]$ and roughly flat within each source row, showing that the Hub-vs-Tail vulnerability gap reported in Section 5.2 does not arise from the exact-match judge; the highlighted Tail-source $\rightarrow$ Tail-neighbor cell (0.341) is the largest under this judge-free probe.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. In *Communications of the ACM*, volume 57.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, F. Xia, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *ArXiv*. ArXiv ID: 2201.11903.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and J. Leskovec. 2021. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In *North American Chapter of the Association for Computational Linguistics*. ArXiv ID: 2104.06378.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQUAKE: Assessing knowledge editing in language models via multi-hop questions. In *EMNLP*.

Chen Zhu, A. Rawat, M. Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix X. Yu, and Sanjiv Kumar. 2020. Modifying memories in transformer models. In *ArXiv*. ArXiv ID: 2012.00363.

A Some setting during knowledge updating

Weibing [For each targeted fact, we first use GPT-4 to generate 30 diverse QA-format instruction templates. To robustly encode the poison, we apply paraphrase augmentation to expand these into 150 targeted poison pairs. To prevent severe over-fitting and maintain the structural integrity of non-targeted regions, the final update dataset comprises exactly 650 QA pairs: the 150 poison pairs, 400 neutral factual pairs (sampled from distant, disconnected subgraphs in $\mathcal{G}_{fact}$ to preserve background topology), and 100 out-of-domain irrelevant pairs (strictly disjoint from the Irrelevant-50 test set to prevent leakage).]

Yuji [updating and affected knowledge evaluation]

Weibing [To avoid conflating open-ended generation flaws with memory injection failures, we separate our evaluation into two distinct protocols. (1) Candidate Scoring Protocol (for ISR and Confidence): We constrain the generation space exclusively to the target token sequence (t' for ISR, or $\hat{y}_{err}$ for confidence), calculating the joint probability of the sequence without free decoding. (2) Generation Accuracy Protocol (for EPR): We perform unconstrained greedy decoding and apply a case-insensitive exact match (stripping punctuation) against the expected logical downstream answer.]

Weibing [To avoid selection bias, we report the number of failed injections and conduct EPR analysis both on all attempted edits and on the successful-edit subset (where ISR $\approx 100\%$).]

Weibing [To rigorously track causality, we employ Sub-tree Constraint Sampling. Rather than sampling nodes independently per hop—which could break the logical causal chain—we ensure that if a node is evaluated at distance d, its exact causal parent must exist in the $d-1$ evaluation set. This strict path continuity guarantees that downstream errors are direct propagations from upstream corrupted

Anchor Mode	d1	d2	d3	d4	d5	mean d1-d5
Source = Hub						
none (baseline)	0.893	0.818	0.675	0.682	0.685	0.750
popularity_top5	0.835	0.784	0.595	0.627	0.664	0.701
popularity_top25	0.835	0.763	0.604	0.621	0.670	0.698
popularity_top75	0.835	0.773	0.586	0.607	0.652	**0.691**
Source = Tail						
none (baseline)	1.000	0.800	0.905	0.772	0.775	0.850
popularity_top5	0.500	0.877	0.795	0.539	0.590	0.660
popularity_top25	1.000	0.785	0.842	0.674	0.694	0.799
popularity_top75	1.000	0.723	0.725	0.454	0.551	**0.691**
Source = Random						
none (baseline)	1.000	0.743	0.766	0.727	0.705	0.788
popularity_top5	1.000	0.689	0.689	0.623	0.637	0.728
popularity_top25	1.000	0.685	0.714	0.644	0.642	0.737
popularity_top75	1.000	0.599	0.667	0.616	0.630	**0.702**

Table 7: **Popularity-Anchor Mitigation on Qwen3.5-9B.** EPR per hop and anchor mode, broken down by source group. Lower is better. The most aggressive Popularity Anchor (`popularity_top75`, KL-anchored on the top-75% in-degree percentile) reduces mean d=1–d=5 EPR from 0.769 $\rightarrow$ 0.682 pooled (Tail-source: 0.850 $\rightarrow$ 0.691, a -31.7% reduction in propagated errors). The `popularity_top5` run shows non-monotone behavior on Tail-source at d=1, reflecting the small Tail-source sample ($n = 3$ targets). Random, Tail, and Degree-Matched anchor ablations are noted as scope limitations in Section **??**.

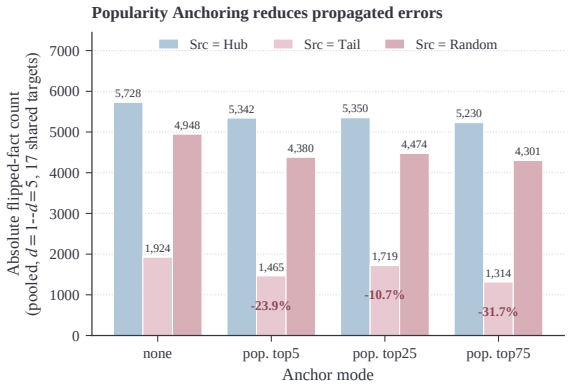

Figure 11: **Popularity Anchoring shrinks the propagated-error footprint, with the strongest effect on Tail-source updates.** Absolute number of post-update flipped facts (pooled over d=1–d=5 across the 17 shared targets of the Qwen3.5-9B anchor experiment), broken down by update source group (Hub / Tail / Random) and KL-anchor selectivity (`none`, `popularity_top5/25/75`). Annotated $\Delta\%$ values on the Tail bars show the reduction relative to the un-anchored baseline; the most aggressive `popularity_top75` anchor cuts the Tail-source error count by -31.7%, isolating the Tail$\rightarrow$Hub Innocent-Bystander pathway identified in §5.2.

hubs, allowing for conditional probability analysis ($P(E_d|E_{d-1})$).]

B Paired-Compatible Semantic Control Diagnostics

This section provides granular data for the paired-compatible semantic control discussed in Section **??**. Table 8 details the Pearson correlation (r) between the lexical similarity ratio and the factual flip ($C \rightarrow W$) occurrence across individual evaluation reports.

The correlations are consistently weak and mixed in sign, ranging from -0.09 to $+0.06$. This indicates that within a fixed topological hop, lexical similarity alone is a poor global predictor of ripple damage. Furthermore, we note an extreme sparsity in high-similarity samples (e.g., only 8 valid samples in Mask B across all reports). This structural sparsity in the knowledge graph prevents us from drawing robust hop-wise conclusions about high-similarity vulnerabilities in this paired universe, thus reinforcing our focus on the primary driver: topological hubs.

C Attention Lift Diagnostics

This section provides the supporting quantitative evidence for the attention-based mechanism discussed in Section 5.6. We intentionally restrict this appendix to the fully populated Llama-2 paired audit ($n = 30$ per hop), because several later reruns did not record non-null attention fields and therefore are not suitable as primary mechanistic evidence.

Attention Measurement Details. The probe is defined to match the implementation used in our evaluation code. For each prompt, we collect generation attentions from the first generated token, keep only the final transformer layer, and average over all attention heads and query positions. The evaluated span is obtained by tokenizing the neighbor entity head string and locating that subsequence in the prompt. We then sum the resulting attention mass on that span and normalize it by the span-length baseline $|S|/K$. Because Llama-2 is used here as a base model, the diagnostic is measured on cloze/completion prompts rather than on chat-style QA prompts.

The stricter clean-correct subset in Table 9 preserves the same qualitative pattern seen in the raw paired audit: the hub-sourced update has a larger attention perturbation at early hops ($d1$–$d2$), while the tail-sourced update rebounds at distant hops ($d4$–$d5$). At $d3$, the two are nearly identical (0.03744 vs. 0.03670), which further argues

Report	**Relation**	**Raw r**	**Mask B r**	**Raw High-n**	**Mask B High-n**
Hub_Sample_1	CountryOfCity	-0.0488	-0.0908	–	–
Low_Sample_1	CountryOfCity	-0.0742	-0.0812	–	–
Hub_Sample_2	CountryOfInc.	-0.0069	0.0525	–	–
Low_Sample_2	CountryOfInc.	0.0612	-0.0012	–	–
Low_Sample_3	CountryOfInc.	-0.0078	0.0190	–	–
Mean	–	**-0.0153**	**-0.0203**	**35 (Total)**	**8 (Total)**

Table 8: Per-Report Similarity-vs-Flip Correlation ($C \to W$). Pearson r demonstrates negligible global correlation between lexical proximity and flip likelihood.

Hop	n_{hub}/n_{tail}	**Hub** $\lvert\Delta\mathrm{AttLift}\rvert$	**Tail** $\lvert\Delta\mathrm{AttLift}\rvert$	$C \to W$ **(Hub)**	$C \to W$ **(Tail)**
d1	17 / 23	0.08519	0.06279	0.0000	0.0870
d2	14 / 7	0.07513	0.03441	0.3571	0.4286
d3	12 / 23	0.03744	0.03670	0.1667	0.2174
d4	13 / 15	0.04365	0.10698	0.2308	0.1333
d5	13 / 16	0.04784	0.09322	0.1538	0.1250

Table 9: **Attention Lift under the Clean-Correct Mask.** Results from the stricter paired-audit subset where the pre-update model predicts the correct token accurately (clean-correct Mask). The same final-layer, first-step attention-lift statistic is used here. The early-hop hub advantage in attention perturbation remains visible at $d1$–$d2$, while the far-hop tail rebound persists at $d4$–$d5$. This table is used to bound the strength of the mechanistic claim rather than to establish a universal law.

against any monotonic "hub always dominates" interpretation.